

Heritage Treasures: An In-Depth Analysis of UNESCO World Heritage Sites in Tableau

Final Project Report

Date	22 July 2026
Team ID	TEAM-HT-2026
Team Lead	Devendra Reddy
Team Member	DHARA MAHITHA

1. Introduction

1.1 Project Overview

Heritage Treasures is a Tableau-based data visualization project built on the UNESCO World Heritage Sites (2019) dataset. It brings together country-level distribution, danger/risk status, and regional inscription trends into a single interactive dashboard and guided story, which is published to Tableau Server/Public and embedded into a Flask web application for easy stakeholder access.

1.2 Purpose

The purpose of this project is to help heritage conservation analysts, government policy makers, tourism boards, and cultural researchers quickly understand which countries hold the most heritage sites, which sites are most at risk, and how conservation/inscription efforts have trended across regions over time — without manually sifting through raw spreadsheet data.

2. Ideation Phase

2.1 Problem Statement

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	a Heritage Conservation Analyst / Policy Maker	identify which countries/regions have the most sites and the highest concentration of at-risk sites	the raw UNESCO dataset is large and unstructured across many attributes	there is no single easy-to-read view of at-risk sites	overwhelmed and unable to prioritize conservation funding
PS-2	a Tourism Board Official / Cultural Researcher	understand how heritage inscriptions have trended over time across regions	inscription dates and regional groupings are buried in raw rows	manually tracking growth patterns year over year is slow and error-prone	unable to confidently identify which regions are increasing conservation efforts

2.2 Empathy Map Canvas

Persona: Heritage Conservation Analyst / Government Policy Maker

Quadrant	Key Points
----------	------------

Says	"I need to know which countries have the most heritage sites at a glance." / "I want to see inscription changes by region."
Thinks	There must be a faster way to compare countries than scrolling through rows.
Does	Downloads the raw CSV and manually filters/counts rows in a spreadsheet.
Feels	Frustrated by manual effort; relieved once a clear visual shows the pattern.
Pains	Data is large and unstructured; no easy way to see year-over-year trends by region.
Gains	An interactive dashboard with country counts, danger split, and a guided story.

2.3 Brainstorming

Ideas were grouped and prioritized as follows:

Idea	Impact	Feasibility	Priority
Treemap of sites by country	High	High	1 – Must Have
Pie chart of sites in danger	High	High	1 – Must Have
Line chart of regional inscription trends	High	Medium	1 – Must Have
Interactive filter panel (Region/Category/Year)	Medium	High	2 – Should Have
Multi-scene guided story	Medium	Medium	2 – Should Have
Flask web app embedding	High	Medium	2 – Should Have

3. Requirement Analysis

3.1 Customer Journey Map

Stakeholders move through five stages when using the dashboard: becoming aware of the published dashboard/story link (Entice), opening it and getting oriented (Enter), exploring the treemap, pie chart, trend line and filters (Engage), drawing conclusions and sharing findings (Exit), and returning later as new data is published (Extend). Positive moments center on fast, clear insight; the main friction point is the initial load time of the embedded Tableau view.

3.2 Solution Requirement

Functional Requirements

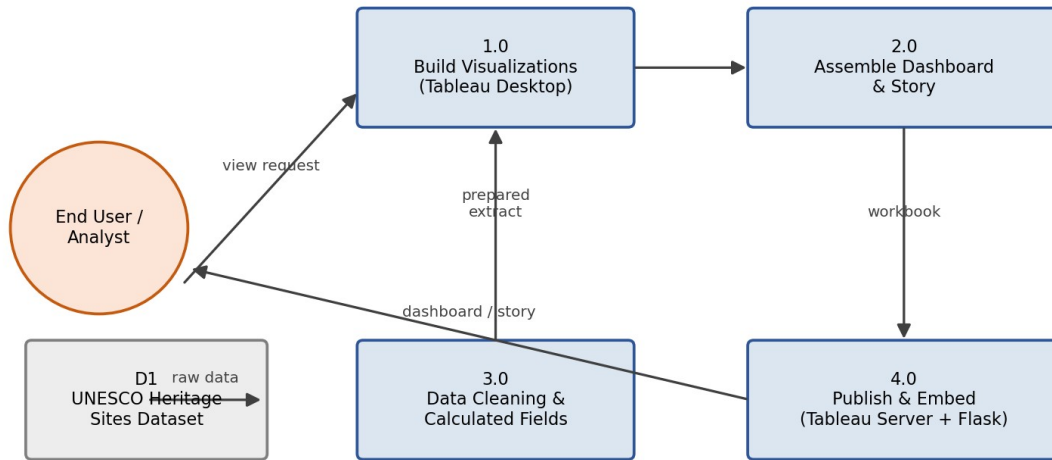
ID	Requirement	Details
FR-1	Data Collection & Extraction	Collect the UNESCO dataset and connect it to Tableau
FR-2	Data Preparation	Clean nulls/duplicates, standardize names, create calculated fields
FR-3	Data Visualization	Treemap by country, danger pie chart, regional trend line
FR-4	Dashboard Design	Combine visuals with interactive filters and cross-filtering
FR-5	Story Creation	Multi-scene guided story with captions
FR-6	Web Integration & Publishing	Publish to Tableau Server/Public and embed in Flask

Non-Functional Requirements

ID	Requirement	Description
NFR-1	Usability	Intuitive for non-technical stakeholders with clear labels and tooltips
NFR-2	Security	Access restricted via Tableau Server/Public permissions; Flask validates input
NFR-3	Reliability	Consistent results with no broken fields or missing data after refresh
NFR-4	Performance	Loads and responds to filters within a few seconds
NFR-5	Availability	Accessible whenever the Flask app and Tableau link are online
NFR-6	Scalability	Accommodates additional years/countries without redesign

3.3 Data Flow Diagram

The Level-0 DFD shows data moving from the raw UNESCO dataset, through cleaning and calculated fields, into Tableau visualizations, and finally to the published dashboard and story that end users interact with.



3.4 Technology Stack

Component	Technology
User Interface	HTML, CSS, Bootstrap, Flask (Jinja templates)
Application Logic	Python (Flask) for routing; Python (Pandas) for data cleaning
Visualization Engine	Tableau Desktop – calculated fields, dashboard, story
Data Source	CSV / Excel – UNESCO World Heritage Sites (2019) dataset
Hosting / Publishing	Tableau Server / Tableau Public
Deployment	Flask app deployed locally or on a cloud host (e.g. Render/Heroku)

4. Project Design

4.1 Problem – Solution Fit

Field	Details
Customer Segment(s)	Heritage conservation analysts, policy makers, tourism boards, researchers
Problem	Cannot quickly see site distribution, at-risk status, or regional trends
Existing Alternatives	Manually filtering the raw CSV in Excel or reading separate static reports
Solution	Interactive Tableau dashboard + story embedded in a Flask web app
Customer Constraints	Users may not know Tableau/SQL – interface must be self-explanatory

Channels of Behaviour	Published Tableau Server/Public link embedded on a web page
-----------------------	---

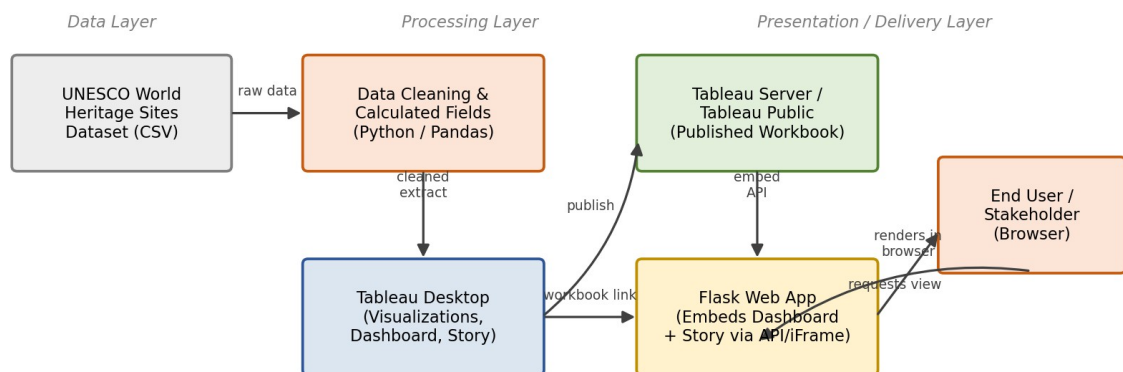
4.2 Proposed Solution

Parameter	Description
Problem Statement	Stakeholders cannot easily see site distribution, danger status or regional trends
Idea / Solution	Treemap + danger pie chart + regional trend line in one dashboard and story
Novelty	Combines distribution, risk and trend views in one filterable dashboard with a narrated story
Social Impact	Helps prioritize conservation funding and highlights strong-performing regions
Business Model	Public-interest/educational tool; extensible into a licensed reporting service
Scalability	Calculated fields and filters allow future data updates without redesign

4.3 Solution Architecture

The architecture spans a Data Layer (raw CSV), a Processing Layer (Python cleaning and Tableau Desktop authoring), and a Presentation Layer (Tableau Server/Public and the Flask web app that end users access).

Heritage Treasures - Solution Architecture



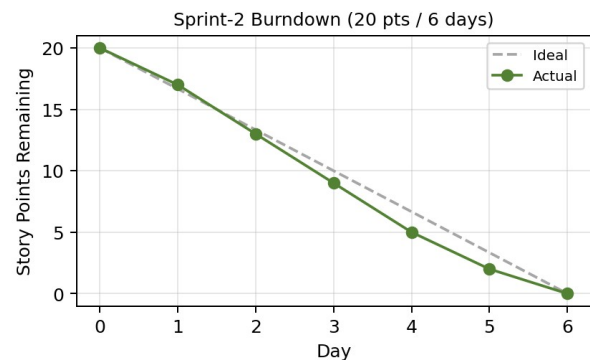
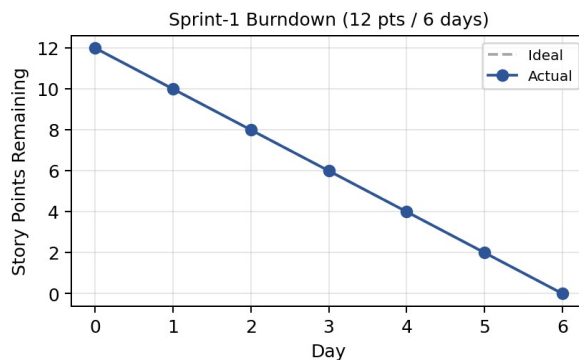
5. Project Planning & Scheduling

5.1 Project Planning

Sprint	Epic	Story Points	Duration
Sprint-1	Data Collection & Extraction, Data Preparation	12	6 Days
Sprint-2	Data Visualization, Dashboard, Story	20	6 Days

Total Story Points = 32 across 2 Sprints → Velocity = $32 / 2 = 16$ Story Points per Sprint.

Heritage Treasures - Sprint Burndown Charts



6. Functional and Performance Testing

6.1 Performance Testing

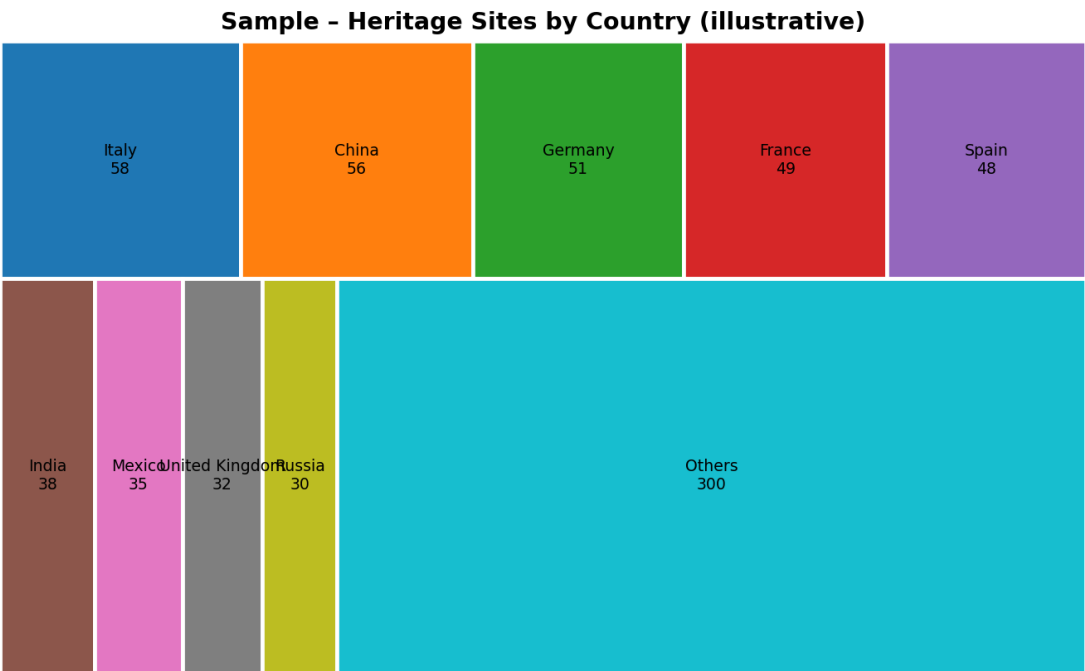
Parameter	Value
Data Rendered	1,121 UNESCO World Heritage Site records
Data Preprocessing	Removed duplicates/nulls, standardized names, derived Danger flag
Utilization of Filters	3 filters – Region, Category, Year range
Calculation Fields Used	4 – Danger Flag, Years Since Inscription, Region Rank, Site Count
Dashboard Design	3 visualizations combined into one responsive dashboard
Story Design	3 story points reusing dashboard worksheets

User Acceptance Testing covered 6 test cases (chart accuracy, filter cross-interaction, story navigation, and the Flask embed) – all passed, with 2 minor low-severity issues logged (embed load-spinner delay, story caption overflow on small screens).

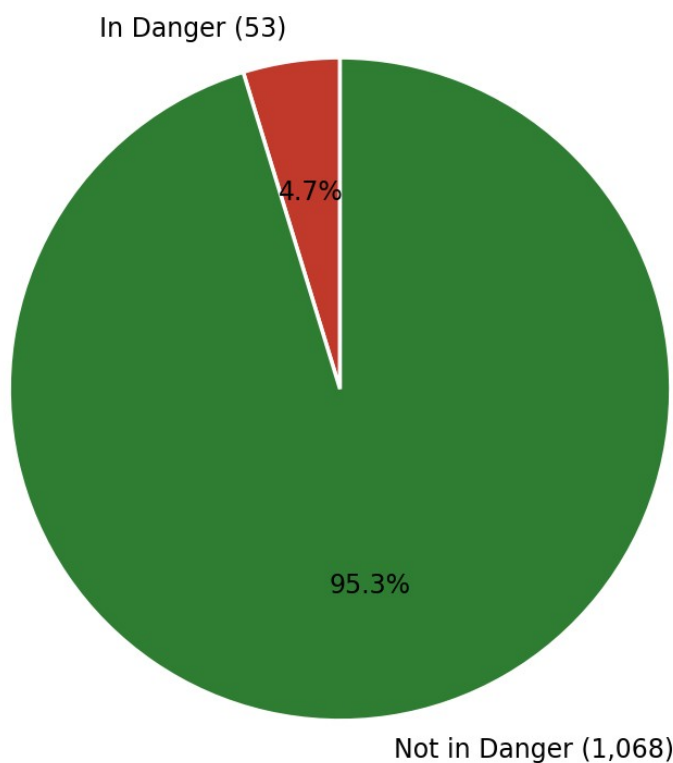
7. Results

7.1 Output Screenshots

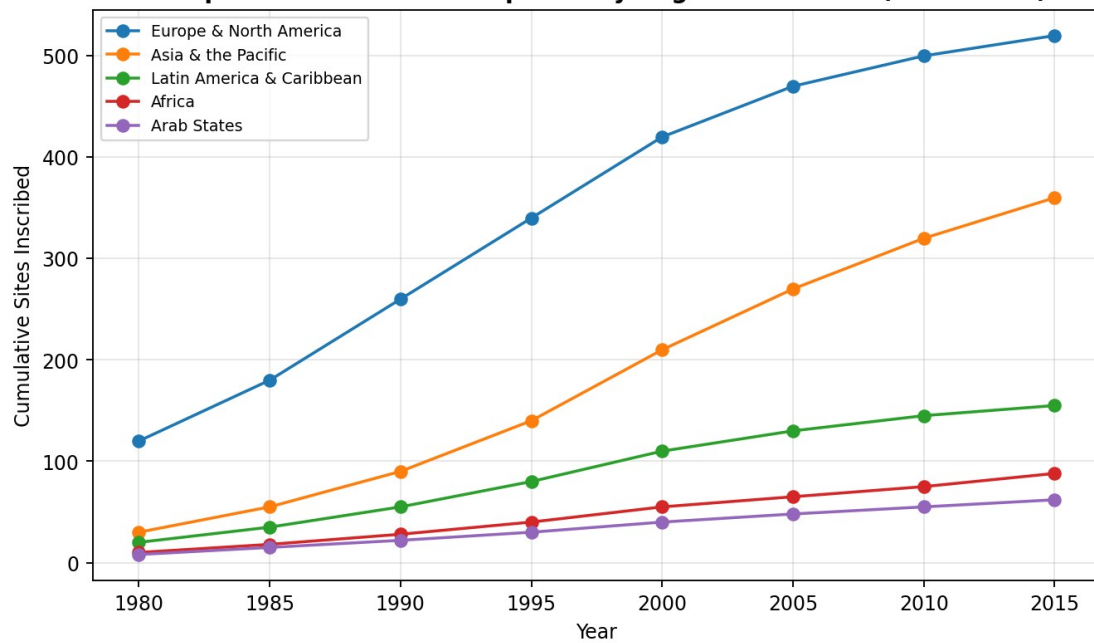
The samples below are illustrative mockups showing the intended look of each visualization with representative sample data. Replace these with actual screenshots once the workbook is published to Tableau Server/Public.



Sample - Sites In Danger vs Not (illustrative)



Sample - Cumulative Inscriptions by Region Over Time (illustrative)



8. Advantages & Disadvantages

Advantages

- Consolidates distribution, risk status and time trends into one interactive view.
- No coding required for end users – filters and tooltips make it self-service.
- Tableau Public/Server hosting keeps infrastructure costs low.
- Story feature makes it easy to communicate insights to non-technical stakeholders.

Disadvantages

- Relies on a static dataset extract – no live/real-time data connection.
- Tableau Public dashboards can be slower to load on first visit.
- Requires Tableau licensing/publishing access for updates beyond the free tier.
- No built-in geographic map view in the current scope.

9. Conclusion

Heritage Treasures successfully turns a large, unstructured UNESCO heritage dataset into a clear, interactive Tableau dashboard and story. By combining a country-wise treemap, an at-risk sites pie chart, and a regional inscription trend line with cross-filtering, the project gives conservation analysts and policy makers a fast, visual way to identify priorities — fulfilling the goals set out in the Ideation and Requirement Analysis phases.

10. Future Scope

- Add a live/incremental data connection instead of a static extract.
- Include a geographic map view plotting sites by latitude/longitude.
- Add a simple '/api/summary' Flask endpoint for programmatic access to aggregate counts.
- Extend the dataset with visitor numbers or funding data for deeper impact analysis.

11. Appendix

Source Code

Source code (Flask app, data cleaning scripts, Tableau workbook) – add your repository path or attach as a separate archive.

Dataset Link

UNESCO World Heritage Sites (2019) dataset – add the exact source link used (e.g. the Kaggle or UNESCO Open Data link) here.

GitHub & Project Demo Link

GitHub Repository: [team lead to add link]

Live Demo: [team lead to add Tableau Public / Flask app link]